Tower of Shadows

Final Game Design Document

A pixel-art top-down dungeon shooter, built in Unity over six labs.

Student: **Mingyu Yang**
Student Number: **3184344**
Module: Games Development (Y4 BSCH Computer Science)
Project Component 4 - Final Design Document
Submission Date: 14 May 2026

Table of Contents

1. Introduction

1.1 Title and Hookline

Title: Tower of Shadows

Hookline: Climb a dark tower, swap between seven different guns, and beat three big bosses.

1.2 Game Overview and Concept

Tower of Shadows is a 2D top-down dungeon shooter for Windows. You play a small pixel character climbing a three-floor tower. Each floor is one scene with several rooms, enemies, weapon chests, and a boss at the end.

Genre: top-down arena shooter. **Setting:** a dark fantasy dungeon with stone rooms and torches on the walls. **Game feel:** fast and punchy. Every shot makes a flash, every kill makes a small particle burst, and every explosion makes a big boom.

Core idea: mix three games I like. Soul Knight for the gun variety and chibi style, Hades for the dark feel, and CS:GO for the simple drop-and-pick-up weapon system. **Target audience:** action game fans aged 13+. **Platform:** Windows PC, keyboard and mouse. **What makes it unique:** almost every asset (sprites, audio, levels, lighting) is built by C# code in the project, so there are no external art or sound files except one music track.

1.3 Inspirations and Influences

Three games and one music library shaped the design:

- **Soul Knight** (ChillyRoom, 2017) gave me the basic loop: top-down view, mouse aim, room-based dungeon, lots of guns, and a chibi pixel style. The 7-weapon set (pistol, machine gun, shotgun, sniper, flame, frost, laser) comes from Soul Knight [1].
- **Hades** (Supergiant Games, 2020) inspired the dark look and the heavy mood of the music. My BGM uses an A-minor drone with a slow heartbeat to copy the feeling of Hades' Tartarus tracks [2].
- **Counter-Strike: Global Offensive** (Valve, 2012) gave me the one-key weapon drop. Like CS:GO, my game uses one button (G) to either swap with a ground gun or drop the current one [3].
- The looping music track *Symmetry* is royalty-free and is credited in the references [4].

2. Gameplay

2.1 Core Gameplay Mechanic

The core mechanic is **twin-stick shooting in closed rooms**. The player moves with WASD and aims with the mouse. The gun rotates on its own so the body always faces the camera while the gun follows the cursor. Left-click fires the current weapon. Ammo drains as you shoot and slowly refills over time. When you walk into a room, the doors close behind you. They open again once every enemy in the room is dead.

This was built in **Lab 2**. The lab asked for basic WASD movement and a shooting prototype. I extended it with mouse aim, a child "GunPivot" transform that turns by itself, and the room-locking system.

2.2 Other Mechanics

- **Seven weapons** - each has its own damage, fire rate, ammo, spread, and effect (burn / slow / pierce / multi-pellet). Added in Lab 3 and extended in Lab 6.
- **Drop and pick up (CS:GO style)** - press G near a weapon to swap; press G in open space to drop. The pistol can never be dropped. Added in Lab 6.
- **Status effects** - flame gun burns enemies over time, frost gun slows them, sniper and laser pierce up to three enemies. Done with coroutines in EnemyBase.cs.
- **Explosive crates** - shoot them to deal area damage to anything in the blast (including you).
- **Health pickups** - enemies sometimes drop a heart that heals you on contact.
- **Player state machine** - tracks Idle / Running / Shooting / Hurt / Dead. Drives the debug panel in the top-left. Added in Lab 4.
- **Boss gate** - the portal to the next floor is locked until the floor boss is dead.
- **Respawn on the same floor** - dying restarts the current floor, not the whole game. Added after Playtest 1.

2.3 Optional Game Elements (Two or More)

My two optional elements are the two Lab 6 features:

- **(1) Lighting Effects (Lab 6)** - a 2D light system using soft radial sprite overlays. There is a warm light around the player, two torches on the wall of every room, a glow around chests, and a coloured halo on dropped guns. The torches and the glow change over time, so they count as *dynamic* lighting.
- **(2) Visual Effects (Lab 6)** - one helper class called ParticleFx fires six different particle bursts: muzzle flash, enemy death, explosion debris, boss ring blast, boss death, and pickup glow. All effects use Unity's built-in ParticleSystem.

Both elements were added in Lab 6 as polish. They do not change the rules of the game, only how it feels to play.

2.4 Game Loop

The basic loop is about 30 seconds long and repeats across all rooms and floors. **Win:** beat the Shadow King on Floor 3. **Lose:** HP drops to zero.

1. **Enter a room** - doors close. You can see enemies thanks to the player's light and the wall torches (Lab 6).
2. **Fight enemies** - move, aim, dodge orbs and charges, shoot. Each shot makes a muzzle flash (Lab 6).
3. **Loot** - dead enemies sometimes drop hearts. Chests drop a new weapon on the floor for you to pick up with G.
4. **Doors open** - kill every enemy and the doors reopen.
5. **Move on** - walk to the next room. When the boss is dead, a portal opens to the next floor.
6. **Win or lose** - beat Floor 3's Shadow King to see the Victory scene. Die at any point to see the death screen with a Restart button that puts you back on the current floor (not Floor 1).

Rewards: new weapons from chests, healing from enemies, and clear audio + particle feedback for every action. **Progression:** Floor 1 -> Floor 2 -> Floor 3 -> Victory, with one boss per floor.

2.5 Scene Transitions

The game uses five Unity scenes. They are linked through one GameManager singleton that survives scene loads using DontDestroyOnLoad. The player can go back to the main menu or restart at any time. On death, the game reloads the current floor automatically.

Scene	Purpose
MainMenu	Title screen. Loads at start and when you return from a run.
Floor1	First level. Four rooms. Boss: Shadow Guardian.
Floor2	Cross-shaped layout. Six rooms. Boss: Shadow Archon.
Floor3	Wide split layout. Seven rooms. Boss: Shadow King.
Victory	End screen with options to play again or go back to the menu.

Transitions are done by GameManager methods: `LoadFloor(int)`, `NextFloor()`, `GameOver()`, `RestartGame()` (reloads the current floor), and `LoadMainMenu()`. The settings panel shows up as an overlay when ESC is pressed instead of being a separate scene.

2.6 User Interface

The HUD was built across Labs 2 to 5. It shows every value the player needs to read at a glance:

Element	Position	Notes
HP bar + number	Top-right	Read from PlayerController.currentHP every frame.
Ammo counter	Bottom-left	Shows "AMMO: 24 / 30". Refills over time.
Weapon name	Bottom-left	Above the ammo, e.g. "Sniper Rifle".

Element	Position	Notes
Floor label	Top-centre	Shows "FLOOR 1 / 3" from GameManager.
State debug panel	Top-left	State / Speed / Direction / Aim / Grounded (Lab 4).
Sound toggles	Bottom-right	"[M] Music: ON" / "[N] SFX: ON". Green if on, red if off (Lab 5).
Interact prompt	Bottom-centre	"Press E to open" or "Press G to swap to [Weapon]".
Boss HP bar	Top-centre	Shows boss name and HP. Only visible during a boss fight.
Pause menu	Full screen	Resume / Restart / Main Menu buttons.
Death screen	Full screen	"YOU DIED" with Restart / Main Menu buttons.

2.7 Sound (Lab 5 deliverable)

This whole system was built in Lab 5. The lab asked for looping music, sound effects on events, and on-screen toggle controls. All three are done.

All audio runs through one AudioManager singleton. It has two AudioSource components - one for music (loop = true) and one for SFX (uses PlayOneShot so effects can overlap). It survives scene loads with DontDestroyOnLoad.

Background music: a looping royalty-free track called *Symmetry* loaded from `StreamingAssets/Audio/bgm.mp3`. It starts when the game starts and plays through every gameplay scene. If the file is missing, the AudioManager builds a dark dungeon track in code (sub-bass drone, minor chord, wind noise, slow heartbeat).

Sound effects (8 in total):

Sound	Trigger Event	Source
Shoot	Left-click fires the gun	Sine sweep, made in code
Hit Enemy	A bullet hits an enemy	Short low tone
Enemy Death	Enemy HP reaches zero	Falling tone
Explosion	Crate is destroyed	**Custom MP3** (the "boom" meme)
Player Hurt	Player takes damage	Noise plus sine
Health Pickup	Player picks up a heart	Rising tone
Door Open	Door opens after a clear	Rising sweep
Chest Open	Player opens a chest	Frequency-sweep chime

Toggle controls: press **M** to mute/unmute the music; press **N** for sound effects. The state is saved in PlayerPrefs, so it stays the same the next time you open the game. The bottom-right of the HUD shows the current state (green = on, red = off).

Bug fix after Playtest 1: A tester said the explosion sound was late. The cause was silence at the start of the mp3 file. I trimmed the file and set its playback volume to 1.4x, so the boom now lands at the same time as the visual.

2.8 Interaction and Controls

All player input is handled in `PlayerController.Update()`. The GameManager handles game-wide hotkeys (ESC, M, N).

Key	Action	Implementation
WASD	Move	Sets velocity on Rigidbody2D
Mouse	Aim	Gun child rotates to face the cursor
Left click	Shoot	Spawns a Bullet prefab + muzzle flash + sparks
E	Open chest	Works when in WeaponChest.interactRange
G	Drop / pick up gun	Single key, CS:GO style. Swap with nearby pickup, or drop in open space (pistol excluded)
ESC	Pause / resume	Shows the pause panel and sets Time.timeScale to 0
M	Toggle music	Saved in PlayerPrefs (Lab 5)
N	Toggle SFX	Saved in PlayerPrefs (Lab 5)

World items the player can interact with: weapon chests (E), weapon pickups (G), explosive crates (shoot), health pickups (walk over), portals (walk into), and doors (open by themselves when the room is clear).

3. Implementation

3.1 Code Structure

The project has 32 C# scripts in seven folders under `Assets/Scripts/`. Each folder has one job:

```
Assets/Scripts/
■■■ Core/ GameManager, AudioManager, CameraFollow
■■■ Player/ PlayerController, PlayerStateMachine, WeaponData,
■ WeaponPickup, Bullet
■■■ Enemies/ EnemyBase, ShadowWalker, ShadowCharger, ShadowCaster,
■ CasterOrb, BossController
■■■ Environment/ WeaponChest, ExplosiveCrate, Portal, HealthPickup,
■ RoomManager, RoomDoor, RuntimeLevelGeometry
■■■ Effects/ SimpleLight, TorchLight, PulsingLight, ParticleFx,
■ FxAssetHolder (Lab 6 polish)
■■■ UI/ UIManager, MainMenuManager, MenuActions
■■■ Editor/ PrototypeAssets, PrototypeBuilder, PrototypeData
```

Design patterns I used:

- **Singleton** - GameManager, AudioManager, UIManager, and PlayerController each have a static Instance field. Core managers also use DontDestroyOnLoad.
- **State Machine** - PlayerStateMachine switches between Idle / Running / Shooting / Hurt / Dead based on a few timers. PlayerController feeds Speed and Direction into the machine every frame.
- **Inheritance with virtual methods** - EnemyBase holds the shared stats, status-effect coroutines, death code, and contact damage. Each enemy subclass (Walker / Charger / Caster / Boss) writes its own Start() and Update(). Bosses also override Die() for extra effects.
- **Static data table** - WeaponData keeps all seven weapon stats in one readonly array. No ScriptableObjects, no extra assets - all weapons live in one file.
- **Static helper class** - ParticleFx exposes Burst() and Flash() as static methods. Any script can fire effects without holding a reference to a manager.

3.2 Scenes

There are five Unity scenes. All five are generated by the menu command `Tower of Shadows -> Prototype -> Generate Content`. The generator (PrototypeBuilder.cs) spawns the camera, audio manager, UI canvas, player, portal, doors, enemies, chests, crates, and lighting for every scene and then saves the scene to disk. The whole game can be rebuilt from one click of a menu item.

PrototypeData.cs holds plain C# data tables (LevelDef, RoomDef, CorridorDef, DoorDef, EnemySpawn, ChestSpawn). Every level is data, not hand-placed objects, so the layouts are version-controlled in source code.

3.3 Events and Prefabs

Prefabs (auto-generated by PrototypeAssets.cs):

- Player.prefab - root + body sprite child + GunPivot/Gun children
- Bullet.prefab - trigger collider + Bullet script

- Orb.prefab - homing caster orb
- HealthPickup.prefab - heart pickup
- Crate.prefab - destructible cover
- ExplosiveCrate.prefab - barrel that explodes
- Portal.prefab - floor exit
- ShadowWalker / ShadowCharger / ShadowCaster.prefab - the three regular enemies
- ShadowKing.prefab - final boss with multi-phase AI
- WeaponChest.prefab - lootable chest

Events: the project uses UnityEvents for UI buttons (wired in PrototypeBuilder via UnityEventTools.AddPersistentListener) and plain C# method calls between singletons. Coroutines drive timed effects (burn DoT, slow, hurt flash). Trigger colliders fire OnTriggerEnter2D for bullet hits, pickups, and portal contact.

Event flow when the player kills an enemy: Bullet.OnTriggerEnter2D fires -> EnemyBase.TakeDamage -> currentHP <= 0 -> EnemyBase.Die() -> AudioManager.PlayEnemyDeath() + ParticleFx.Burst() + (sometimes) drop a heart + RoomManager.OnEnemyDied(). When the last enemy in the room dies, RoomManager opens the doors.

3.4 Level Design (built across Labs 1-4)

The three floors were designed to feel *different*, not just bigger:

Level	Shape	Notes
Floor 1	Diagonal path, 4 rooms	Spawn -> RoomA -> RoomB -> Boss room. A linear teaching level: one chest, simple Walker and Charger enemies, ends with the charging Shadow Guardian.
Floor 2	Cross-shaped hub, 6 rooms	Spawn at the bottom -> Hub in the centre -> Left and Right branches -> Top -> Boss at the top. Adds ranged Caster enemies. Two chests reward exploration.
Floor 3	Wide split, 7 rooms	Spawn on the left -> top and bottom paths -> they meet -> large boss room on the right. Mixes all three enemy types and ends with the multi-phase Shadow King.

Each level is defined in PrototypeData.cs. A LevelDef has its scene name, the player spawn point, the portal spawn, an array of rooms (RectInt areas with a key string), corridors between them, doors that close when the player enters, enemy spawns tagged by room, chests with WeaponType indices, and crate positions. The PrototypeBuilder reads this data and builds a wired Unity scene from it. The same builder code handles all three floors.

Lab 6 lighting placement is also automatic: two torches sit on the inner top wall of every room (at 25% and 75% of the width). Each room ends up with two visible light sources, but the dungeon still feels dark overall.

Screenshots: in-game captures of each floor, a chest opening, the muzzle flash, the Shadow King's ring blast, and a death respawn are in the gameplay demo video bundled with this submission.

4. Process

4.1 Scope, Risks and Concerns

Initial scope (from the Concept document): one procedurally-generated dungeon level, one boss, three enemy types, three weapons. Playable in five minutes.

How the scope changed: once the prototype was running, the lab schedule pushed it past the original plan. Each lab added a feature on top of the previous one:

- **Lab 1-2:** player movement, camera follow, basic shooting. Core loop in place.
- **Lab 3:** three enemy AI types (Walker / Charger / Caster) with status effects (burn / slow / pierce).
- **Lab 4:** full UI (HP, ammo, weapon, floor, state debug, pause / death menus) and a state machine driving the debug panel.
- **Lab 5:** the whole audio system - looping BGM, 8 sound effects, M/N toggle keys with PlayerPrefs.
- **Lab 6:** the lighting system (SimpleLight / TorchLight / PulsingLight), six particle effects, and the CS:GO-style weapon drop / pickup.

Risks and problems I ran into:

- **No URP:** Unity's built-in 2D lighting needs the Universal Render Pipeline. Switching pipelines mid-project was risky. *Fix:* built a sprite-based fake-light system in Lab 6 that works in any pipeline.
- **Audio format problem:** Unity's runtime audio loader does not support .m4a. *Fix:* AudioManager warns about unsupported formats and falls back to procedural audio. I converted the source files to .mp3.
- **Subclass reset boss HP:** ShadowCharger / ShadowCaster / ShadowWalker Start() set maxHP to a fixed default and silently overwrote my boss values. *Fix:* the Start() now only sets defaults when maxHP is still the EnemyBase fallback (20).
- **Demo video too big:** my first recording was 680 MB (Moodle limit is 500 MB). *Fix:* re-encoded with ffmpeg at 1080p / CRF 22, down to about 50 MB.

4.2 Technologies and Components Needed

- **Unity 6 (6000.0+)** - game engine.
- **2D built-in render pipeline** - SpriteRenderer, Tilemap, CompositeCollider2D.
- **Physics2D** - Rigidbody2D for the player, enemies, bullets, orbs, and crates.
- **ParticleSystem** (Unity built-in) - drives every visual effect through ParticleFx.
- **UnityWebRequest + StreamingAssets** - async load of the .mp3 BGM and the boom sound.
- **UnityEditor + AssetDatabase** - the PrototypeAssets / PrototypeBuilder scripts run in the editor to build sprites, prefabs, scenes, and tilemaps.
- **PlayerPrefs** - saves the music / SFX toggle state between sessions.
- **ffmpeg** (outside Unity) - used to re-encode the demo video for submission.

5. Playtests

Two playtests were run during Semester 2, with three testers each. Tracey (lecturer), Bernie (classmate) and Chengrui (classmate) played through Floor 1 fully and most of Floor 2.

Playtest 1 (16 April 2026) - Key findings

- All three testers said **dying sends you back to Floor 1** was the biggest pain point.
- Tracey said the **player moves too slowly** to dodge boss attacks.
- Both Tracey and Bernie wanted **better boss fights** - the old boss had 300 HP and a single attack.
- Bernie noticed the **explosion sound was late**.
- Chengrui asked for **more visual variety** between enemies - everyone looked like a circle.
- All three said the **sound design was their favourite element**.

Changes I made after Playtest 1

- **Respawn on the same floor** - dying now reloads the current floor (GameManager.RestartGame).
- **Boss rework** - HP raised to 400 / 550 / 1000 across the three floors. Contact damage doubled. Multiple attack phases. New 12-orb ring blast on Floor 3.
- **Trimmed the explosion mp3** and bumped its volume to 1.4x. The boom now lands at the same time as the visual.
- **New sprite art** - drew a chibi pixel player (orange / blue / black hair), a purple ghost Walker, a red horned Charger, a blue hooded Caster, and the gold-crown Shadow King.
- **Lab 6 polish** - torches, pickup glows, particles. They cover the dark dungeon and give the combat more impact.

Playtest 2 (30 April 2026) - Confirmation

The second playtest, after Lab 5 (sound) and Lab 6 (effects) were finished, confirmed the changes worked. No tester reported the "back to Floor 1" issue. The lighting and particles were called out as standout features. The Shadow King's ring blast was described as "the most satisfying attack to dodge."

What I learned

The main lesson: *polish is not optional*. The highest-scored features in tester feedback (sound, weapon variety, "funny / creative") were the polished ones. The lowest-scored elements (boss fights, enemy art) were the unpolished ones. The lab schedule was timed perfectly - Lab 5 and Lab 6 were both polish labs, and they fixed exactly the issues from Playtest 1.

The full playtest reports (GD_PlayTest_Report_1.pdf, GD_PlayTest_Report_2.pdf) are in the zip with this document.

6. Conclusion

What went right

- **Lab-by-lab progress worked.** Treating each lab as a milestone (Lab 5 = sound, Lab 6 = effects) kept scope under control and gave me a visible improvement every two weeks.
- **Generating assets in code saved time.** Because every sprite, prefab, scene, and audio clip is built by C# at editor time, rebuilding the game after a change takes one click.
- **Lighting and particles changed the feel.** The same fights felt flat in Lab 4 and cinematic in Lab 6, after I added the lighting overlay and ParticleFx.
- **Playtest feedback drove real decisions.** Boss difficulty, respawn behaviour, and enemy art were all changed based on tester comments.

What went wrong

- **The first boss was too weak.** A bug in ShadowCharger.Start() reset maxHP at runtime, so the boss was no harder than a regular enemy until I fixed it.
- **Audio format compatibility** wasted a session. The .m4a file I tried to use kept failing; converting to .mp3 solved it.
- **Demo video was too big.** Recording at 1440p / 60fps / high bitrate gave me a 680 MB file. I had to re-encode it.

Did I fulfil my original goals?

Yes. The concept document said "a small but polished pixel-art dungeon shooter". The final build delivers that, plus more: three floors instead of one, seven weapons instead of three, a real audio system with toggles, atmospheric lighting and particles, and a CS:GO-style weapon economy that was not in the original plan. The only goal that slipped was "procedurally generated dungeons". I switched to hand-designed levels because hand-tuned rooms felt more memorable in playtests.

What I learned

- **Trust the lab schedule.** Each lab fixed a real problem in my game. Sound (Lab 5) and polish (Lab 6) were not just exercises - they fixed the exact complaints from Playtest 1.
- **Singleton + state machine + static data was enough.** I didn't need DI containers, event buses, or ScriptableObjects. Three patterns covered every architectural need.
- **Procedural assets save weeks.** Drawing pixel art in C# meant I could change a character in seconds instead of editing PNGs.
- **Playtest feedback is gold.** Every change between Playtest 1 and the final build came from a tester comment.

What I would improve with more time

- **Enemy animations** - frame-based idle / walk / attack instead of single-frame sprites.
- **More weapons and consumables** - bombs, grenades, dash boots.
- **Save system** - save player progress, unlocked weapons, and best clear times.

- **Procedural levels** - generate rooms instead of hand-defining them, for replay value.
- **Controller support** - current scheme is keyboard + mouse only.
- **Music layering** - the BGM should get more intense during boss fights.

7. References

[1] ChillyRoom, 2017. *Soul Knight* [video game]. ChillyRoom Inc. Available at: https://chillyroom.com/en (Accessed: 12 March 2026).

[2] Supergiant Games, 2020. *Hades* [video game]. Supergiant Games. Available at: https://www.supergiantgames.com/games/hades (Accessed: 12 March 2026).

[3] Valve, 2012. *Counter-Strike: Global Offensive* [video game]. Valve Corporation. Available at: https://www.counter-strike.net (Accessed: 12 March 2026).

[4] MacLeod, K. *Symmetry* [music]. incompetech.com. Licensed under Creative Commons: By Attribution 4.0 License. Available at: https://incompetech.com/music/royalty-free/index.html (Accessed: 25 April 2026).

[5] Unity Technologies, 2026. *Unity Manual*. Unity Technologies. Available at: https://docs.unity3d.com (Accessed: 12 March 2026).

[6] Unity Technologies, 2026. *Scripting API - MonoBehaviour, Rigidbody2D, ParticleSystem, AudioSource, PlayerPrefs*. Available at: https://docs.unity3d.com/ScriptReference (Accessed: 12 March 2026).

[7] FFmpeg developers, 2025. *FFmpeg 8.1.1* [software]. Available at: https://ffmpeg.org (Accessed: 6 May 2026). Used outside Unity to re-encode the gameplay demo video.

[8] Bilibili community, 2024. *"Boom" meme sound effect*. Used as a trimmed mp3 for the explosive crate sound.